DEADLOCK AVOIDANCE

Aim-Implement Algorithm to solve bankers algorithm

Theory –

Deadlock avoidance is one of the three main strategies for handling **deadlock** in an operating system, alongside prevention and detection/recovery. It is a set of techniques used to ensure the system will *never* enter an **unsafe state**—a state from which deadlock is inevitable. Deadlock avoidance requires more information about process resource needs than detection, but is less restrictive than prevention.

Core Concept: Safe State

The entire theory of deadlock avoidance revolves around the concept of a **safe state**:

- **Safe State:** A system state is considered safe if there exists a **safe sequence** of all processes. A safe sequence $\langle P_1, P_2, \dots, P_n \rangle$ is an order in which every process P_i can request its remaining needed resources and then finish. When P_i finishes, it releases its allocated resources, which can then be used by the next process P_{i+1} in the sequence. If the system can always find such a sequence, it can guarantee that no process will be perpetually blocked (deadlocked).
- **Unsafe State:** A state that is not safe. An unsafe state **may** lead to a deadlock, but it doesn't guarantee one immediately. However, since the system cannot guarantee that all processes will finish, the avoidance algorithm treats it as a dangerous state to be avoided.

The Avoidance Strategy

The operating system must dynamically check the resource-allocation state to ensure that a circular wait condition can never arise.

1. **Advance Declaration:** Every process must declare the **maximum number of resources** of each type it may request before it begins execution.
2. **Resource Request Check:** When a process requests a set of resources, the system must determine if granting the request would leave the system in a safe state.
3. **Conditional Grant:**
    - If the resulting state is **safe**, the resources are allocated to the process.
    - If the resulting state is **unsafe**, the resources are withheld, and the requesting process must wait until other processes release sufficient resources.

Banker's Algorithm:

The Banker's Algorithm is a **deadlock avoidance algorithm** proposed by Edsger Dijkstra. Its primary goal is to ensure that a system remains in a **safe state**—a state where the operating system can guarantee that all processes will eventually complete their execution without entering a deadlock. The algorithm models the operating system as a banker, the processes as customers, and resources as currency (hence the name). A process must state its **maximum resource needs** upfront. Before granting any resource request, the algorithm simulates the allocation to check if the new state remains *safe*. If granting the request leads to an unsafe state (where the system cannot find a sequence for all processes to finish), the request is denied, and the process is made to wait. This preemptive check is what makes it an **avoidance** algorithm, as opposed to detection or prevention methods.

Key Matrice

The Banker's Algorithm relies on four main matrices to track the system's resource status. Let n be the number of processes and m be the number of distinct resource types.

1. Available Vector (or Matrix)

- **Size:** $1 \times m$
- **Purpose:** This vector shows the **number of instances of each resource type currently available** in the system for immediate use.
- **Example:** If $m=3$ (Resources A, B, C), and Available = $[3, 2, 2]$, it means there are 3 instances of A, 2 instances of B, and 2 instances of C that are unallocated and ready to be granted.

2. Max Matrix

- **Size:** $n \times m$
- **Purpose:** This matrix defines the **maximum demand** of each process for each resource type. This is the largest total amount of resources a process may request throughout its entire execution.
- **Example:** If Process P_1 has Max $[1] = [5, 4, 3]$, it means P_1 will never need more than 5 units of A, 4 units of B, and 3 units of C combined (allocated + requested).

3. Allocation Matrix

- **Size:** $n \times m$
- **Purpose:** This matrix shows the **number of resources of each type currently held** (allocated) by each process.
- **Example:** If Process P_1 has Allocation $[1] = [1, 2, 1]$, it means P_1 is currently holding 1 unit of A, 2 units of B, and 1 unit of C.

4. Need Matrix

- **Size:** $n \times m$

- **Purpose:** This matrix shows the **remaining resources each process still needs** to complete its task. This is the difference between the **Max** demand and the **Allocation** it currently holds. This is the amount the process may request in the future.

{Need}[i, j] = {Max}[i, j] - {Allocation}[i, j]$$

- **Example:** If a process P_1 has:
    - text{Max}[1] = [5, 4, 3]$
    - {Allocation}[1] = [1, 2, 1]$
    - Then, {Need}[1] = [5-1, 4-2, 3-1] = **[4, 2, 2]**$. P_1 still needs 4 A, 2 B, and 2 C to complete.

The **Safety Algorithm** uses the Available and Need matrices to find a **safe sequence** of processes.

EXAMPLE –

Banker's algorithm

Available: A = 3, B = 3, C = 5

Process	**Allocation A**	**Allocation B**	**Allocation C**	**Max A**	**Max B**	**Max C**	**Need A**	**Need B**	**Need C**
P1	1	1	3	5	2	6	4	1	3
P2	0	1	1	3	2	1	3	1	0
P3	1	2	0	6	3	1	5	1	1
P4	2	0	1	2	0	3	0	0	2

Safe-sequence computation

Start with Work = Available = (A,B,C) = (3,3,5).

- P2's Need = (3,1,0) ≤ Work (3,3,5) ⇒ grant P2. New Work = Work + Allocation(P2) = (3,4,6).
- P4's Need = (0,0,2) ≤ Work (3,4,6) ⇒ grant P4. New Work = (5,4,7).
- P1's Need = (4,1,3) ≤ Work (5,4,7) ⇒ grant P1. New Work = (6,5,10).
- P3's Need = (5,1,1) ≤ Work (6,5,10) ⇒ grant P3. New Work = (7,7,10).

Safe sequence found: P2 → P4 → P1 → P3.

.

CODE –

```
#include <iostream>

#include <iomanip>

#include <vector>

#include <cstdlib>

#include <string>


using namespace std;


int m, n;

vector<vector<int>> Allocation, Max, Need;

vector<int> Available;


int safety() {

  vector<int> safeSequence(n, -1), Work = Available;

  vector<bool> Finish(n, false);


  int k = 0;

  bool found;


  cout << "\nSafety Algorithm Execution:\n";

  cout << "Initial Work: ";

  for (int j = 0; j < m; j++) cout << Work[j] << " ";

  cout << endl;


  while (true) {

    found = false;


    for (int i = 0; i < n; i++) {

      if (!Finish[i]) {

        bool canAllocate = true;

        for (int j = 0; j < m; j++) {

          if (Need[i][j] > Work[j]) {
```

```
                canAllocate = false;
                break;
            }
        }

        if (canAllocate) {
            for (int j = 0; j < m; j++)
                Work[j] += Allocation[i][j];
            Finish[i] = true;
            safeSequence[k++] = i;
            found = true;

            cout << "Allocated P" << i << ", Work: ";
            for (int j = 0; j < m; j++) cout << Work[j] << " ";
            cout << ", Sequence so far: ";
            for (int idx = 0; idx < k; idx++)
                cout << "P" << safeSequence[idx] << (idx < k - 1 ? " " : "");
            cout << endl;
        }
    }
  }

  if (!found)
    break;
}

for (int i = 0; i < n; i++) {
  if (!Finish[i]) {
    cout << "\nUnsafe state.\n";
    return 1;  // unsafe
  }
}

cout << "\nSafe state. Sequence: <";
```

```
    for (int j = 0; j < n; j++)
        cout << "P" << safeSequence[j] << (j < n - 1 ? ", " : ">\n");

    return 0; // safe
}

void display() {
    const int procWidth = 8;
    const int resWidth = 12;  // Adjust as needed for better alignment

    cout << left << setw(procWidth) << "Process" << "| " << left << setw(resWidth) << "Allocated" << "| " << left << setw(resWidth) << "Maximum" << "| " << left << setw(resWidth) << "Need" << endl;
    cout << string(procWidth, '-') << "|-" << string(resWidth, '-') << "|-" << string(resWidth, '-') << "|-" << string(resWidth, '-') << endl;

    for (int i = 0; i < n; i++) {
        string allocStr = "";
        for (int j = 0; j < m; j++) {
            allocStr += to_string(Allocation[i][j]) + (j < m - 1 ? " " : "");
        }
        string maxStr = "";
        for (int j = 0; j < m; j++) {
            maxStr += to_string(Max[i][j]) + (j < m - 1 ? " " : "");
        }
        string needStr = "";
        for (int j = 0; j < m; j++) {
            needStr += to_string(Need[i][j]) + (j < m - 1 ? " " : "");
        }

        cout << left << setw(procWidth) << ("P" + to_string(i)) << "| " << left << setw(resWidth) << allocStr << "| " << left << setw(resWidth) << maxStr << "| " << left << setw(resWidth) << needStr << endl;
    }

    cout << string(procWidth, '-') << "|-" << string(resWidth, '-') << "|-" << string(resWidth, '-') << "|-" << string(resWidth, '-') << endl;
```

```
    string availStr = "";
    for (int j = 0; j < m; j++) {
        availStr += to_string(Available[j]) + (j < m - 1 ? " " : "");
    }
    cout << left << setw(procWidth) << "Available" << "| " << left << setw(resWidth) << availStr << "| " << string(resWidth, ' ') << "| " << string(resWidth, ' ') << endl;
}

void displayTemp() {
    const int numWidth = 3;
    int allocWidth = m * numWidth;
    int availWidth = m * numWidth;
    int needWidth = m * numWidth;
    int procWidth = 7;

    cout << left << setw(procWidth) << "Process" << " | " << left << setw(allocWidth) << "Allocation" << " | " << left << setw(availWidth) << "Available" << " | " << left << setw(needWidth) << "Need" << endl;

    cout << string(procWidth, '-') << "|" << string(allocWidth, '-') << "|" << string(availWidth, '-') << "|" << string(needWidth, '-') << endl;

    for (int i = 0; i < n; i++) {
        cout << left << setw(procWidth) << ("P" + to_string(i)) << " | ";
        for (int j = 0; j < m; j++) cout << right << setw(numWidth) << Allocation[i][j];
        cout << " | ";
        if (i == 0) {
            for (int j = 0; j < m; j++) cout << right << setw(numWidth) << Available[j];
        } else {
            cout << string(availWidth, ' ');
        }
        cout << " | ";
        for (int j = 0; j < m; j++) cout << right << setw(numWidth) << Need[i][j];
        cout << endl;
    }
```

```
    cout << string(procWidth, '-') << "|" << string(allocWidth, '-') << "|" << string(availWidth, '-') << "|" <<
string(needWidth, '-') << endl;
}

int main() {
    cout << "Enter the number of Processes: ";
    cin >> n;
    cout << "Enter the number of Resource types: ";
    cin >> m;

    Allocation.assign(n, vector<int>(m, 0));
    Max.assign(n, vector<int>(m, 0));
    Need.assign(n, vector<int>(m, 0));
    Available.assign(m, 0);

    cout << "\nEnter Allocated Resources for each process:" << endl;
    for (int i = 0; i < n; i++) {
        cout << "P" << i << ": ";
        for (int j = 0; j < m; j++)
            cin >> Allocation[i][j];
    }

    cout << "\nEnter Max Resources for each process:" << endl;
    for (int i = 0; i < n; i++) {
        cout << "P" << i << ": ";
        for (int j = 0; j < m; j++)
            cin >> Max[i][j];
    }

    cout << "\nEnter Available Resources:" << endl;
    for (int i = 0; i < m; i++)
        cin >> Available[i];
```

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        Need[i][j] = Max[i][j] - Allocation[i][j];

display();
int stateFlag = safety();

char choice = 'y';
do {
    vector<int> Request(m);
    int p;

    cout << "\n\nEnter Process Number: ";
    cin >> p;

    if (p < 0 || p >= n) {
        cout << "\nInvalid process number.\n";
        cout << "Try another? (Y/N): ";
        cin >> choice;
        continue;
    }

    cout << "Enter Request: ";
    for (int i = 0; i < m; i++)
        cin >> Request[i];

    bool allZeroes = true;
    for (int i = 0; i < m; i++) {
        if (Request[i] != 0) {
            allZeroes = false;
            break;
        }
    }
```

```
if (allZeroes) {
    cout << "\nCannot grant - request is all zeros.\n";
    cout << "Try another? (Y/N): ";
    cin >> choice;
    continue;
}

bool invalid = false;
for (int i = 0; i < m; i++) {
    if (Request[i] > Need[p][i]) {
        cout << "\nProcess exceeded maximum claim for resources.\nRequest Cannot be granted" << endl;
        invalid = true;
        break;
    } else if (Request[i] > Available[i]) {
        cout << "\nProcess must wait. Resources not available." << endl;
        invalid = true;
        break;
    }
}

if (invalid) {
    cout << "Try another? (Y/N): ";
    cin >> choice;
    continue;
}

for (int i = 0; i < m; i++) {
    Available[i] -= Request[i];
    Allocation[p][i] += Request[i];
    Need[p][i] -= Request[i];
}

cout << "\nTemporary State:" << endl;
displayTemp();
```

```
        stateFlag = safety();

        if (stateFlag) {
            cout << "\nRequest cannot be granted" << endl;
            for (int i = 0; i < m; i++) {
                Available[i] += Request[i];
                Allocation[p][i] -= Request[i];
                Need[p][i] += Request[i];
            }
            cout << "\nStates Restored:" << endl;
            display();
        } else {
            cout << "\nSafe Sequence Exists and request can be granted immediately to process.\nSnapshot after
request:\n";
            display();
        }

        cout << "\n\nTry another Process?(Y/N)";
        cin >> choice;

    } while (choice == 'y' || choice == 'Y');

    return 0;
}
```

```
Enter the number of Processes: 4
Enter the number of Resource types: 3

Enter Allocated Resources for each process:
P0:
1 1 3
P1: 0 1 1
P2: 1 2 0
P3: 2 0 1

Enter Max Resources for each process:
P0: 5 2 6
P1: 3 2 1
P2: 6 3 1
P3: 2 0 3

Enter Available Resources:

3 3 5
Process | Allocated    | Maximum      | Need
--------|-------------|-------------|-------------
P0      | 1 1 3        | 5 2 6        | 4 1 3
P1      | 0 1 1        | 3 2 1        | 3 1 0
P2      | 1 2 0        | 6 3 1        | 5 1 1
P3      | 2 0 1        | 2 0 3        | 0 0 2
--------|-------------|-------------|-------------
Available| 3 3 5       |              |

Safety Algorithm Execution:
Initial Work: 3 3 5
Allocated P1, Work: 3 4 6 , Sequence so far: P1
Allocated P3, Work: 5 4 7 , Sequence so far: P1 P3
Allocated P0, Work: 6 5 10 , Sequence so far: P1 P3 P0
Allocated P2, Work: 7 7 10 , Sequence so far: P1 P3 P0 P2

Safe state. Sequence: <P1, P3, P0, P2>


Enter Process Number: 1
Enter Request: 3 0 3

Process exceeded maximum claim for resources.
Request Cannot be granted
Try another? (Y/N): N
```

Code –

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#define BUFFER_SIZE 5

#define ITEM_SIZE 10

char buffer[BUFFER_SIZE][ITEM_SIZE];

int in = 0, out = 0;

int empty = BUFFER_SIZE;

int full = 0;

int mutex = 1;


void wait(int *s) {

  if (*s > 0)
```

```
        (*s)--;
    else
        printf("Wait failed: semaphore value is zero.\n");
}

void signal(int *s) {
    (*s)++;
}

void produce(char item[]) {
    wait(&empty);
    wait(&mutex);
    strcpy(buffer[in], item);
    printf("\nAdded: %s at position %d\n", item, in);
    in = (in + 1) % BUFFER_SIZE;
    signal(&mutex);
    signal(&full);
}

void consume() {
    wait(&full);
    wait(&mutex);
    printf("\nRemoved: %s from position %d\n", buffer[out], out);
    strcpy(buffer[out], "_");
    out = (out + 1) % BUFFER_SIZE;
    signal(&mutex);
    signal(&empty);
}

void displayBuffer() {
    int i;
```

```
    printf("\nBuffer Status:\n");
    printf("--------------\n");
    for (i = 0; i < BUFFER_SIZE; i++) {
        printf("Slot %d: %s\n", i, buffer[i]);
    }   printf("--------------\n");  printf("Empty: %d, Full: %d, In: %d, Out: %d\n", empty, full, in, out);
}
int main() {
    int choice;
    char item[ITEM_SIZE];
    for (int i = 0; i < BUFFER_SIZE; i++)
        strcpy(buffer[i], "_");

    while (1) {
        printf("\n--- MENU ---\n");
        printf("1. Produce Item\n");
        printf("2. Consume Item\n");
        printf("3. Display Buffer\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                if (empty == 0)
                    printf("\nOverflow\n");
                else {
                    printf("Enter item: ", ITEM_SIZE - 1);
                    scanf("%s", item);
                    produce(item);
                }
                break;
```

```
        case 2:
          if (full == 0)
            printf("\nUnderflow\n");
          else
            consume();
          break;
        case 3:
          displayBuffer();
          break;
        case 4:
          printf("\nExiting...\n");
          exit(0);
        default:
          printf("\nInvalid choice\n");      }
    }
    return 0;
}
```

OUTPUT –

```
--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1
Enter item: Banana

Added: Banana at position 0

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1
Enter item: Apple

Added: Apple at position 1

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 3

Buffer Status:
--------------
Slot 0: Banana
Slot 1: Apple
Slot 2: _
Slot 3: _
Slot 4: _
--------------
Empty: 3, Full: 2, In: 2, Out: 0

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1
Enter item: Mango

Added: Mango at position 2

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1
Enter item: Orange

Added: Orange at position 3
```

```
--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1
Enter item: Peach

Added: Peach at position 4

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 3

Buffer Status:
--------------
Slot 0: Banana
Slot 1: Apple
Slot 2: Mango
Slot 3: Orange
Slot 4: Peach
--------------
Empty: 0, Full: 5, In: 0, Out: 0

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 1

Overflow

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Removed: Banana from position 0

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Removed: Apple from position 1

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Removed: Mango from position 2
```

```
--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Removed: Orange from position 3

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Removed: Peach from position 4

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Underflow

--- MENU ---
1. Produce Item
2. Consume Item
3. Display Buffer
4. Exit
Enter choice: 2

Underflow
```

Conclusion – Algorithm to solve BANKERS ALGORITHM problem was implemented successfully.